# Principal Component Analysis (PCA)

DAEN 300 - Fall 2026



# Assignment

- The assignment is due next Friday before class.
- It's not graded, just marked completed as satisfactory/unsatisfactory, so don't stress.
- That said, use it to study, because we'll start class with a short, multiple-choice quiz on the same material.
- Be sure to arrive on time so you don't miss any of the quiz!

**IMPORTANT**

Heads up $\rightarrow$ Fri Nov 20 and Tue Dec 1 (redefined day), we'll build a mini project in class. Please make sure you don't miss these sessions! If you miss either session, you'll make it up on final exam day. If you attend both, you're done: Dec 1 will be our last class!

# Why PCA?

- Real data often has **many columns**: sensors, pixels, metrics.
- Many columns tell the same story. They go up and down together.
- PCA replaces them with a **few new columns** that keep most of the information.

**What it is used for:**

- Shrinking data (64 columns $\rightarrow$ 2)
- Plotting data that has too many columns to draw
- Removing noise
- Preparing inputs for a model

PCA does not use labels. It only looks at the numbers.

Students usually struggle with PCA in three areas:

- **Interpreting** the results
- The **mathematical intuition** behind the method
- Knowing **when to apply** the technique

# 1. What do the Principal Components (PCs) actually mean?

This is the biggest difficulty. Because PCA creates new variables as **linear combinations** of the original ones, the original physical meaning is lost.

### The question

If PC1 is a mix of *age*, *salary*, and *credit score*, how do I explain to the client or the business what PC1 represents?

### The answer

Look at the **loadings** of each original variable on the component. If *salary* and *credit* have very high weights on PC1, students learn to interpret PC1 informally as **"Purchasing Power."**

# 2. How many Principal Components should I choose?

## The question

If I had 20 variables and PCA produced 20 components, how many should I keep in my model?

## The answer

Students often confuse the different criteria. The usual approaches are:

- The **elbow plot** (*scree plot*)
- Keeping the components that together account for **70%–90% of the total explained variance**

# 3. Why is it mandatory to standardize (scale) the data?

## The question

What happens if I forget to use `StandardScaler` (in Python)?

## The answer

PCA is extremely **sensitive to scale**.

- Height in meters: roughly 1 to 3
- Salary: roughly \$2,000 to \$10,000

PCA will put almost 100% of its attention on the variable with larger numbers (salary), treating it as having more variation and ignoring the importance of the other. Standardizing gives every variable **mean 0 and variance 1**.

# 4. Does PCA work for categorical variables (text/classes)?

## The question

Can I apply PCA to columns such as *Gender*, *Marital Status*, or *Car Type*?

## The answer

**Not directly.** Traditional PCA was designed strictly for **continuous numerical data**, since it relies on covariance and Euclidean distance.

For categorical data, students find they need variant techniques such as:

- **Multiple Correspondence Analysis (MCA)**
- **Factor Analysis of Mixed Data (FAMD)**

# 5. Does PCA help predict better? Is it a predictive model?

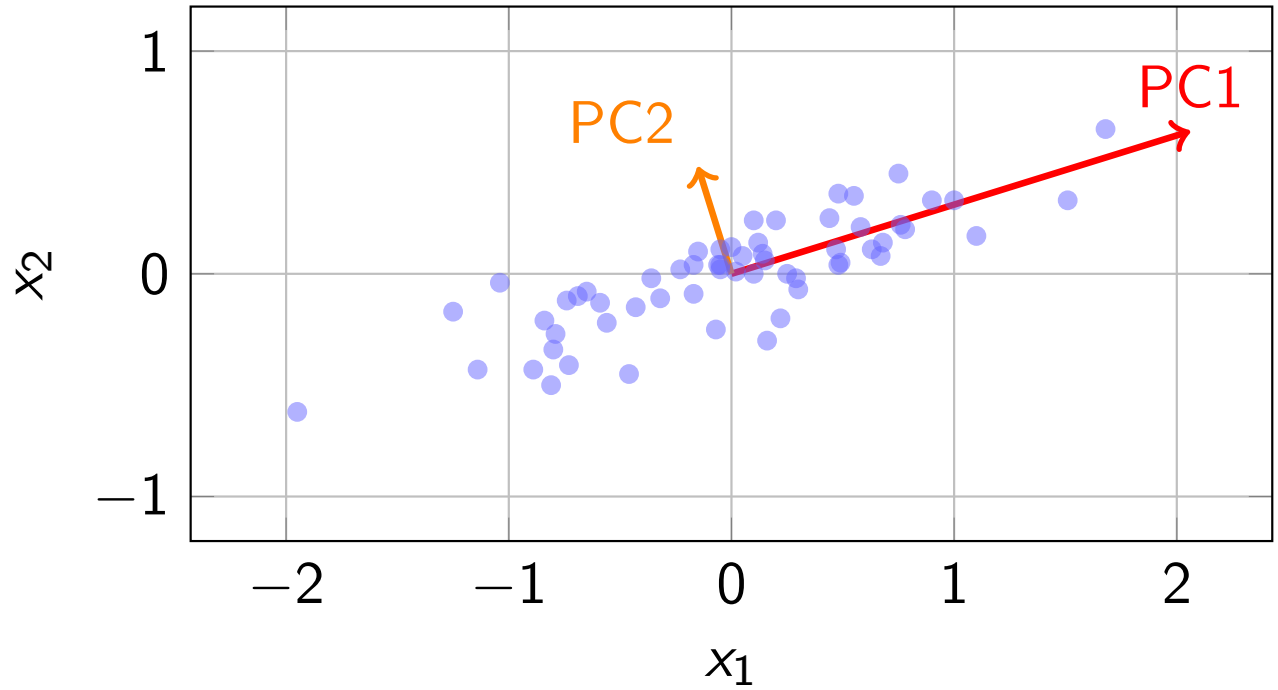
## The question

Can I use PCA to predict the price of a house?

## The answer

**No.** PCA does not predict anything on its own; it is an **unsupervised learning** algorithm.
It only *tidies up the house*: it cleans and compresses the data so that another algorithm (such as a **Regression** or a **Decision Tree**) can make the prediction faster and more efficiently.

# The idea in one picture




- The points lie close to a line.
- **PC1** points along that line, where the data spreads the most.
- **PC2** is at a right angle to PC1 and holds the small leftover spread.
- We can describe each point mostly by *where it falls along PC1*.

# The recipe

1. **Center**: subtract each column's average.
2. **Covariance**: measure how the columns spread and move together.
3. **Eigenvectors and eigenvalues** of the covariance matrix:
   - eigenvector = a new direction (an axis)
   - eigenvalue = how much the data spreads along it
4. **Sort** the directions, biggest spread first.
5. **Project**: find each point's position along the new axes. Keep only the first few.

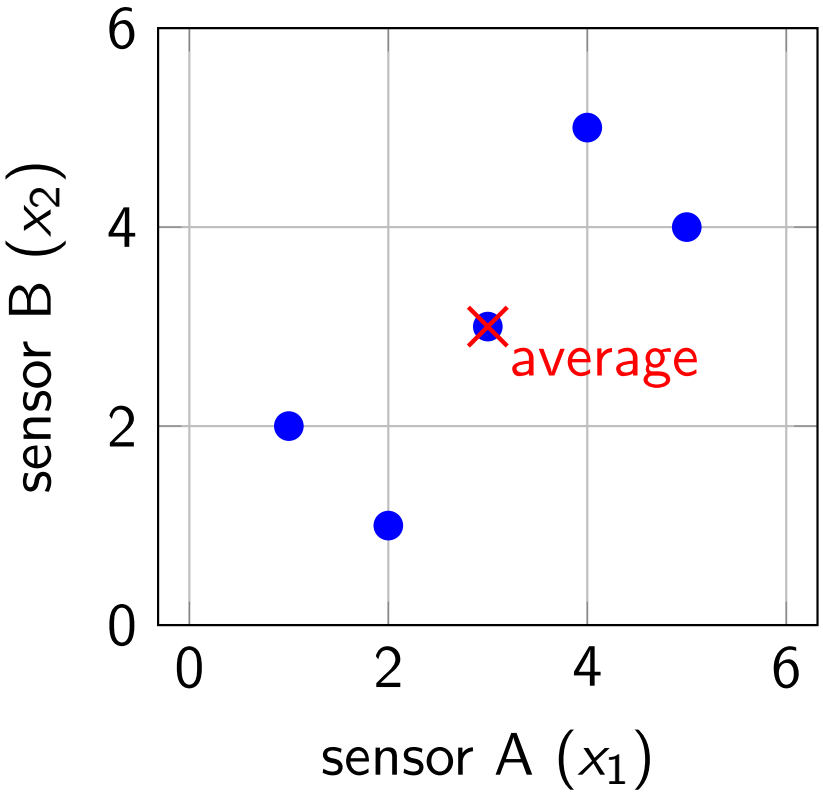
$$C = \frac{1}{n-1} X_c^\top X_c \qquad\qquad C\,v = \lambda\,v$$

# Example: two temperature sensors



Two sensors on the same machine, five readings.

| Reading | $x_1$ | $x_2$ |
|---|---|---|
| 1 | 1 | 2 |
| 2 | 2 | 1 |
| 3 | 3 | 3 |
| 4 | 4 | 5 |
| 5 | 5 | 4 |
| Average | 3 | 3 |




The sensors mostly agree: when A reads high, B reads high too.

# Steps 1–2: center and covariance



**Center:** subtract the average $(3, 3)$ from every reading.

$$X_c = \begin{pmatrix} -2 & -1 & 0 & 1 & 2 \\ -1 & -2 & 0 & 2 & 1 \end{pmatrix}^\top$$

**Covariance:** multiply, add up, divide by $n - 1 = 4$.

$$\text{spread of } x_1 = \frac{4+1+0+1+4}{4} = 2.5$$

$$\text{spread of } x_2 = \frac{1+4+0+4+1}{4} = 2.5$$

$$\text{moving together} = \frac{2+2+0+2+2}{4} = 2.0$$

$$C = \begin{pmatrix} 2.5 & 2.0 \\ 2.0 & 2.5 \end{pmatrix} \qquad \text{total spread} = 2.5 + 2.5 = 5.0$$

# Step 3: eigenvalues and eigenvectors



Solve $\det(C - \lambda I) = 0$:

$$(2.5 - \lambda)^2 - 2^2 = 0 \;\Rightarrow\; 2.5 - \lambda = \pm 2 \;\Rightarrow\; \boxed{\lambda_1 = 4.5, \;\; \lambda_2 = 0.5}$$

The matching directions (length 1):

$$v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \qquad v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

**Check:** $C v_1 = \frac{1}{\sqrt{2}}(4.5,\ 4.5)^\top = 4.5\, v_1$ ✓

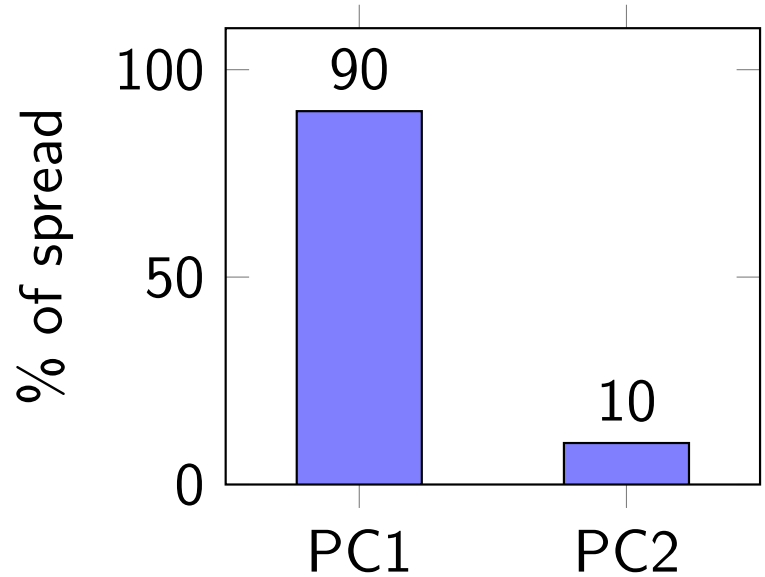
- $4.5 + 0.5 = 5.0$: the same total spread, just on new axes.
- PC1 $(1, 1)$: both sensors together, i.e. the *shared* temperature.
- PC2 $(1, -1)$: how much the sensors *disagree*.

# How much does each direction hold?



Share of the total spread:

| | $\lambda$ | $\frac{\lambda_k}{\lambda_1 + \lambda_2}$ share |
|---|---|---|
| PC1 | 4.5 | **90%** |
| PC2 | 0.5 | 10% |




One number per reading (instead of two) keeps 90% of the information. This is called the **explained variance ratio**.

# Steps 4–5: new coordinates



Each reading's position along the new axes (its **score**):

$$z_1 = \frac{x_{c1} + x_{c2}}{\sqrt{2}} \qquad z_2 = \frac{x_{c1} - x_{c2}}{\sqrt{2}}$$

| Reading | centered | $z_1$ | $z_2$ |
|---|---|---|---|
| 1 | $(-2, -1)$ | $-2.12$ | $-0.71$ |
| 2 | $(-1, -2)$ | $-2.12$ | $0.71$ |
| 3 | $(\ 0,\ 0)$ | $0.00$ | $0.00$ |
| 4 | $(\ 1,\ 2)$ | $2.12$ | $-0.71$ |
| 5 | $(\ 2,\ 1)$ | $2.12$ | $0.71$ |

**Check:** spread of $z_1$ is 4.5 and spread of $z_2$ is 0.5 (eigenvalues ✓)

Software may flip the sign of a direction. The scores flip too; nothing else.

# Keep only PC1

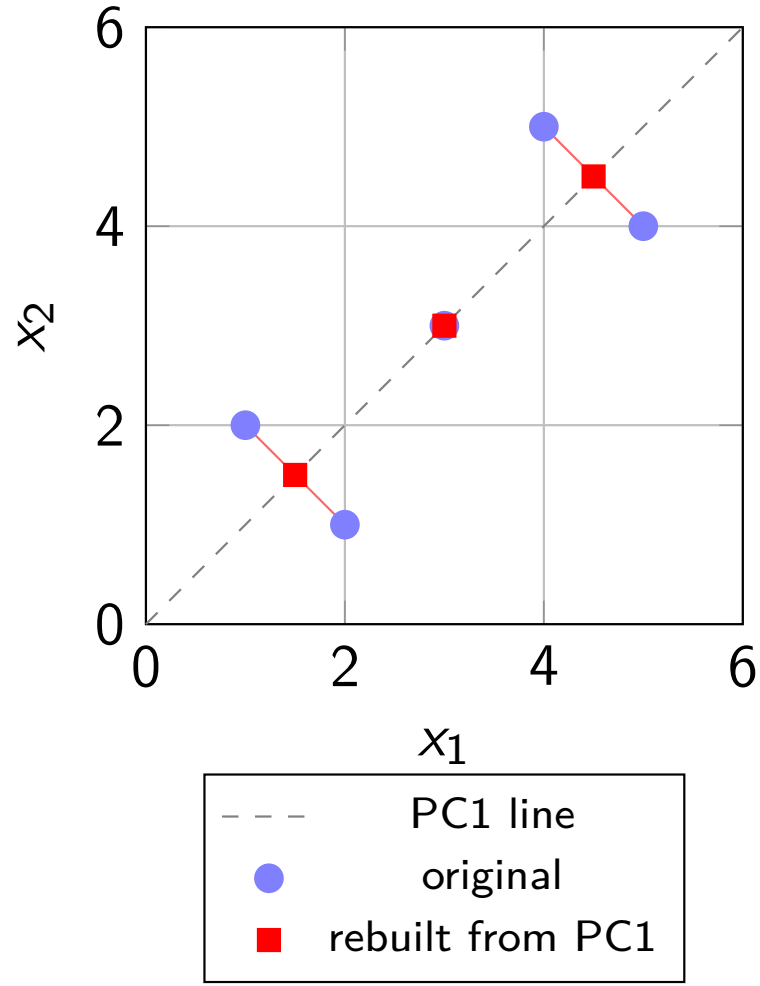


Drop $z_2$ and rebuild each reading from $z_1$ alone:

$$\hat{x} = \text{average} + z_1 \, v_1$$

Reading 1:
$(3, 3) + (-2.12)(0.707, 0.707)$
$= (3, 3) + (-1.5, -1.5)$
$= (1.5,\ 1.5)$

$(1, 2)$ becomes $(1.5, 1.5)$. The disagreement is gone; the shared reading stays.



# The same thing in scikit-learn



```
import numpy as np
from sklearn.decomposition import PCA

X = np.array([[1, 2], [2, 1], [3, 3], [4, 5], [5, 4]])

pca = PCA(n_components=2).fit(X)
pca.mean_                       # [3, 3]
pca.components_                 # directions v1, v2 (one per row)
pca.explained_variance_         # [4.5, 0.5]
pca.explained_variance_ratio_   # [0.9, 0.1]

pca1 = PCA(n_components=1).fit(X)
Z = pca1.transform(X)           # the z1 scores, shape (5, 1)
X_hat = pca1.inverse_transform(Z)  # rebuilt readings
```

- `fit`: learn the directions from the data.
- `transform`: get the scores.
- `inverse_transform`: go back to the original columns.

# The book example: 200 points



- 200 random points in 2 columns that move together.
- PC1 direction $\approx (0.944,\ 0.329)$
- Spread along each direction $\approx (0.763,\ 0.018)$

Share kept by PC1:

$$\frac{0.763}{0.763 + 0.018} \approx 0.976$$

PC1 alone keeps **about 98%** of the information. Going from 2 columns to 1 loses very little.

The book shows 0.759 instead of 0.763. Older scikit-learn divided by $n$, current versions divide by $n - 1$ (slide 6). The 98% is the same either way.

# Handwritten digits



- 1,797 images, each $8 \times 8$ pixels = **64 numbers** per image.
- Each number says how dark one pixel is.
- `PCA(2)` turns the 64 numbers into 2, so each image becomes a dot on a plot.

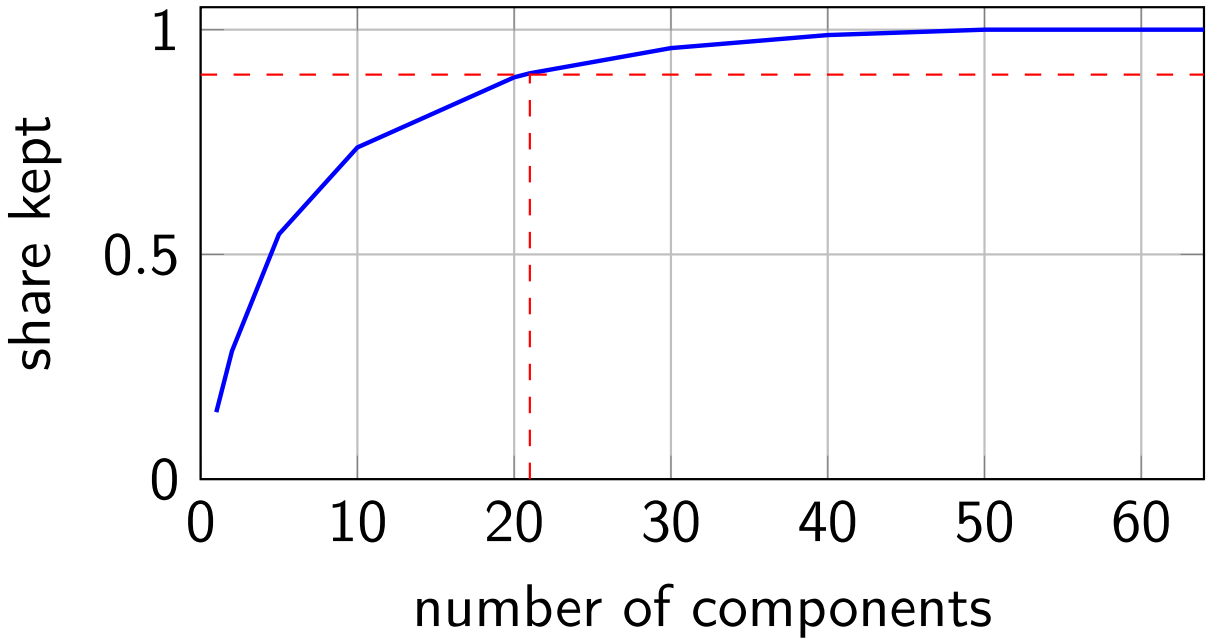
**Why the dots form groups:**

- Two "0"s darken almost the same pixels, so their 64 numbers are similar.
- Similar numbers $\Rightarrow$ similar PC scores $\Rightarrow$ nearby dots.
- Digits that share strokes (3, 5, 8) land close and overlap.

The 2 components keep only about **28%** of the information, but that is enough to see the groups. PCA never saw the labels.

# How many components to keep?






| Components | 2 | 10 | 21 | 50 |
|---|---|---|---|---|
| Share kept | 28% | 74% | 90% | ≈100% |

Pick the number where the curve reaches your target, e.g. 90%.
`PCA(0.90)` picks it for you (here: 21).

# PCA as a noise filter



**Idea:** the real pattern sits in the first few components. Random noise is spread thinly over all of them.

1. Add random noise to the digit images.
2. Keep the components that hold 50% of the information: `PCA(0.50)` keeps **12**.
3. Rebuild the images from those 12 $\Rightarrow$ cleaner digits.

Same as the sensors: dropping PC2 removed the disagreement, which is where sensor noise shows up.

A model trained on 12 columns instead of 64 is faster and less bothered by noise.

# Eigenfaces



- Face photos of $62 \times 47$ pixels = about **2,900 numbers** each.
- Keep only the first **150** components (about 20 times smaller).
- They hold just over **90%** of the information.
- The rebuilt faces are still easy to recognize.
- First components capture lighting; later ones capture eyes, nose, mouth.

Storage per face: $2{,}914 \times 4$ bytes $\approx 11.7$ KB vs. $150 \times 4 \approx 0.6$ KB.

The book uses `RandomizedPCA`. In current scikit-learn write `PCA(150, svd_solver="randomized")`.

# Watch out: units and outliers



Same five sensor readings, two common problems.

**Units.** Record sensor B in mK instead of K (numbers 1000× bigger).

- PC1 becomes $\approx (0, 1)$: PCA just follows the bigger numbers.
- **Fix:** standardize each column first (`StandardScaler`). PC1 goes back to $(0.71, 0.71)$ and 90%.

**Outliers.** Add one faulty reading, $(5, -10)$.

- PC1 changes from (0.71, 0.71) to $(-0.10, 1.00)$.
- One bad point turned "both sensors" into "sensor B only".
- **Fix:** find and remove or flag bad readings before PCA.

# Other limits

- **Straight lines only.** PCA finds straight directions. Data that curves needs other methods.
- **New columns are mixes.** PC1 is a blend of the original columns. Look at `components_` before giving it a name.
- **Large data.** `PCA(svd_solver="randomized")` finds the first few components quickly.

# Summary



1. PCA finds the directions where the data spreads the most.
2. Steps: center, covariance, eigenvectors, project.
3. Eigenvector = direction; eigenvalue = spread along it.
4. The share of spread tells you how much you keep.
5. Uses: shrinking data, plotting, removing noise, preparing model inputs.

# How many components does PCA produce?

If you start with **10 original variables** (10 columns in your table), PCA will produce **exactly 10 principal components (PCs)**.

There is one component for each original dimension of your data space.

## The crucial detail

What makes PCA useful is that the components are **not equally important**.

# The difference between original variables and PCs

The number is the same, but the distribution of the "information" (**variance**) changes completely.

## Before PCA: original variables

The information is **spread across all columns**.

- Column 1 holds a little, column 5 holds a little more...
- Many columns **repeat information** from one another (they are correlated).

## After PCA: principal components

PCA arranges the new axes in **order of relevance**.

- Each new axis carries as much of the remaining information as possible.
- The first PCs hold most of the information.

# How the information is ordered across the PCs

- **PC1** captures the largest possible share of all the information in the dataset.
- **PC2** takes as much as possible of what is left.
- **PC3** takes a little more...
- ...and so on, down to the last PCs (such as **PC9** or **PC10**), which hold almost no relevant information and carry only **mathematical noise**.

# Why does this matter? (The key trick)

Because the last components are almost useless, **you can throw them away**.

Example

- 10 original variables $\rightarrow$ PCA produces 10 PCs.
- You may find that **PC1, PC2, and PC3 together already hold 92%** of all the information in your dataset.
- So you **discard the other 7 axes**.

This is how a complex **10-dimensional** problem becomes a simple **3-dimensional** one **without losing the essence of your data**.